**Warden**

TECHNICAL REFERENCE



# Warden

A checkpoint-native migration guard for Databricks Delta tables. When a migration fails, Warden rolls the table back through Delta time-travel *and* explains what the migration was trying to achieve — using intent recorded in an Entire checkpoint, never a guess reconstructed from a diff.

Built for the Bengaluru Tech Week Buildathon, Entire Track 1 (Checkpoint-Native Developer Experience), with Best Use of Databricks opted in. Every figure in this document is real output captured from a live warehouse — nothing here is illustrative.

## 1 · What Warden is

A migration fails. The standard response is a blind revert: the schema returns to its previous state, and every trace of *why* the change was attempted goes with it. The engineer handling the incident — often not the person who wrote the migration — reconstructs intent from a commit message under time pressure.

Warden closes that gap. It reads the recorded intent for a change *before* touching the warehouse, applies the migration, validates it, and on failure restores the table through Delta time-travel while surfacing that recorded intent to the operator. The rollback primitive itself is ordinary; what is not ordinary is that the recovery can say what was being attempted.

> THE DISTINCTION THAT MATTERS
>
> **Checkpoint-driven, not checkpoint-logged.** Many tools write a checkpoint as a side effect of doing work. Warden *reads* one to make the recovery useful. Remove Entire and the feature does not degrade — it disappears.

SCOPE, STATED UP FRONT

One Delta table, one migration, one validation rule, one healing path. This is a deliberately narrow vertical slice proven end-to-end, not a general migration framework. Section 10 lists what that excludes.

---

## 2 · Quick start

### Verify without any credentials

Both of these run on a clean checkout with no Databricks account, and are the fastest way to confirm the repository is intact:

```
# fixture + seed integrity check — exits non-zero if anything is broken
bash warden/demo.sh --self-test

# unit suite (18 pass, 3 live-integration tests skip cleanly)
python3 -m pytest warden/ databricks/ -q
```

### Run the live demo

```
# 1. credentials — gitignored .env, never committed
echo 'DATABRICKS_SERVER_HOSTNAME=...' >> .env
echo 'DATABRICKS_HTTP_PATH=...'       >> .env
echo 'DATABRICKS_TOKEN=...'           >> .env
set -a; source .env; set +a

# 2. provision the table and synthetic seed data
python3 -c "from databricks import sql; import os; c=sql.connect(
  server_hostname=os.environ['DATABRICKS_SERVER_HOSTNAME'],
  http_path=os.environ['DATABRICKS_HTTP_PATH'],
  access_token=os.environ['DATABRICKS_TOKEN']); cur=c.cursor();
  cur.execute('CREATE SCHEMA IF NOT EXISTS warden');
  [cur.execute(s) for s in open('databricks/setup.sql').read().split(';') if s.strip(
python3 databricks/seed.py

# 3. the full fail → heal loop
bash warden/demo.sh
```

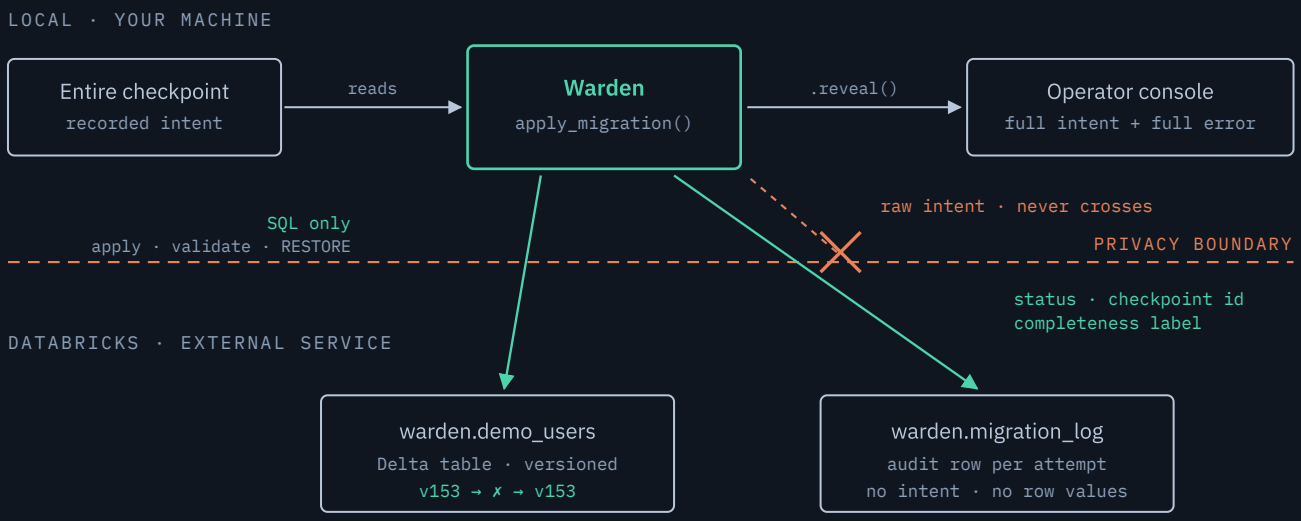
FREE EDITION CAVEAT

A Databricks Free Edition warehouse spins down when idle and can take 30–60s to cold-start. If the demo appears to hang on its first query, that is the warehouse waking, not a failure. `warden/demo-recording.txt` holds a captured successful run for exactly this situation.

---

## 3 · Architecture

One command drives the whole loop: `python3 warden/migrate.py <migration.sql> --validate-sql <check.sql>`. Everything below happens inside `apply_migration()` (`warden/migrate.py:246`).




Delta owns the restore. Entire owns the reasoning. Warden joins them — and decides what is allowed to cross.

The recorded intent is read and printed locally in full; only a status, a checkpoint *id*, and a completeness label cross into Databricks. The blocked arrow is the mechanism: the intent has a path to the operator and no path to the warehouse.

Reading the diagram, in four points:

- **Left to right is the read path.** Warden pulls recorded intent from the checkpoint before touching the warehouse, and prints it in full to the operator — the only place the raw text ever appears.
- **Top to bottom is the write path.** Two things cross into Databricks and nothing else: the SQL that applies, validates and restores; and one audit row per attempt.
- **The struck arrow is the point.** Raw intent has a path to the console and no path to the warehouse — and that is enforced by the type (`LocalOnlyText`), not by a reviewer remembering the rule.
- **The version line under the Delta table is the proof.** `v153 → ✗ → v153`: same version before and after, so the heal genuinely happened rather than being reported.

01 **Read recorded intent**

`get_latest_checkpoint()` lists checkpoints as JSON, filters to *committed* ones only, and explains the most recent. In-progress shadow checkpoints are skipped deliberately — their intent can still change, so they are not settled evidence.

---

02 **Impact analysis, before the change**

`graph_impact()` shells out to `entire graph impact`. On timeout or error it returns an explicit `(graph impact unavailable: …)` string rather than silently skipping — graph output is evidence, never asserted as fact.

---

03 **Record the pre-migration version, then apply**

`current_delta_version()` reads `DESCRIBE HISTORY … LIMIT 1`. That version number is the rollback target, captured before any statement executes.

---

04 **Validate — from either of two sources**

The supplied validation query runs as an independent check. *In this demo it is never reached:* Delta validates existing rows at `ADD CONSTRAINT` time, so the `ALTER TABLE` itself throws first. The healing logic catches a failure from either source, because not every migration uses a Delta `CHECK`.

---

05 **Heal**

`RESTORE TABLE … TO VERSION AS OF <pre_version>`. This is a metadata operation — it repoints the transaction log rather than rewriting data, which is why it is viable on large tables.

---

06 **Explain locally, log safely**

`describe_failure()` returns a pair: `console_text` carrying the full intent and error for the operator, and `db_note` carrying neither for the warehouse. See section 4.

### Failure modes and how each is handled

| CONDITION | BEHAVIOUR | LOGGED STATUS |
|---|---|---|
| Validation passes | Migration stands | `applied` |
| Migration or validation fails, rollback succeeds | Table restored to pre-version | `healed` |
| Rollback *itself* fails | Error re-raised; both exception types logged, neither message | `failed` |
| Version lookup fails before apply | Rollback skipped — no safe target exists — and reported as such | `failed` |
| Entire unavailable / checkpoint missing | Proceeds with `context_completeness = unavailable` | unchanged |
| Databricks env vars missing | Fails closed with a named list of what is absent | — |

The third row is the genuinely unrecoverable case: the table is left un-restored. It is recorded as `failed` and never as `healed`, because misreporting an un-restored table as healed is the

worst available outcome. That path is covered by a dedicated test.

---

## 4 · A migration fails at 2am

The architecture above is easier to judge against a real situation. Here is the one Warden is built for — a plan-normalisation constraint that meets data predating the rule. Every command and every line of output below is real.

**WITHOUT WARDEN**

- **02:04** — the migration job fails. An alert fires with a stack trace.
- **02:06** — is the table half-changed? Nobody is sure. Someone checks manually.
- **02:15** — a revert is applied. The schema is back. *Why* the change was attempted is gone.
- **09:30** — the author explains it in standup. The constraint was right; the data was older than the rule.
- **Next sprint** — someone re-runs the same migration, because nothing recorded why it failed.

**WITH WARDEN**

- **02:04** — the migration fails and the table is already restored to v153.
- **02:05** — the log row says `healed`, names the checkpoint, and labels the context `complete`.
- **02:06** — the on-call engineer reads the recorded intent locally: normalise plan values *before* enforcing the rule.
- **02:20** — a corrected migration is written: backfill first, then constrain.
- **—** the failed attempt stays in `migration_log` as a queryable record.

### Step by step

**1** **The change is proposed — and its reasoning is captured for free**

A developer, or an agent, works on the migration. Entire records the session as a checkpoint automatically; nobody writes documentation. The recorded intent is a by-product of doing the work.

```
$ git commit -m "enforce supported plan values"
  → Entire-Checkpoint: 01M1TXWCSHSCXDB1KVNT03JB8E
```

---

**2** **Warden reads that intent *before* it touches anything**

This ordering is the whole design. Intent is retrieved while the system is still healthy, so it is available later when the situation is not. The blast radius is checked at the same time.

```
$ bash warden/demo.sh

┌─ 1 · BEFORE
│  Reading the current Delta version — this is the rollback target.
└

   Delta version 153
```

## 3 Delta rejects it — server-side, not by a check we wrote

The constraint validates *existing* rows at `ADD CONSTRAINT` time. One seeded row (`user007`, `plan='legacy'`) predates the rule, so the `ALTER TABLE` itself throws. The failure is Delta's verdict, not our approximation of one.

```
[warden] FAILURE: [DELTA_NEW_CHECK_CONSTRAINT_VIOLATION] 1 rows in
         workspace.warden.demo_users violate the new CHECK constraint
         (plan IN ('free', 'pro', 'enterprise'))
```

## 4 The table heals itself

Warden restores to the version captured in step 2. Metadata-only — the transaction log is repointed, no data is rewritten, so table size does not change the cost.

```
[warden] healing: restoring 'warden.demo_users' to Delta version 153 via time-tra

     v153  ──►  ✗ constraint violated  ──►  v153
   ✓ VERSIONS MATCH — the table was genuinely restored, not just reported.
```

## 5 The operator gets the reasoning; the warehouse gets the label

Locally, the full recorded intent prints — enough to write a corrected migration. In Databricks, the same event becomes an audit row carrying a status, the checkpoint *id*, and a completeness label. The reasoning and the record are deliberately not the same artifact.

```
              status  healed
       checkpoint_id  01M1TXWCSHSCXDB1KVNT03JB8E
                note  migration failed (ServerOperationError); rolled back
```

```
                        warden.demo_users to Delta version 153. Checkpoint
                        intent kept local-only (context: complete).
context_completeness    complete
```

---

### 6 Anyone picking this up asks one question first

Not "what happened" but "do I have enough context to act". That question has a command, and it answers without exposing the checkpoint text to whoever is asking.

```
$ python3 warden/resume_report.py 01M1TXWCSHSCXDB1KVNT03JB8E

=== Warden safe resume report ===
Context completeness: complete
Latest migration log: status=healed, context=complete
Recommended next action: review locally — latest migration healed before retrying
```

> WHAT ACTUALLY CHANGED
>
> The table is safe in both columns — Delta would have restored it either way. What differs is that the failed attempt left behind a *usable* artifact: the reasoning reached a human in minutes instead of at standup, and the record of the attempt outlived the incident.

### Why living inside Databricks matters here

A detail that is easy to skim past: `warden.migration_log` is an ordinary Delta table. It is not a proprietary log format, not a hosted service, and not a new tool anyone has to adopt. That has consequences that a bespoke store would not give you:

- **Every Databricks tool already works on it.** SQL, Lakeview dashboards, alerts, scheduled queries, Genie — none of them need to know Warden exists. Migration health is queryable by whoever already queries the warehouse.
- **It can be joined against the data it describes.** `migration_log` and `DESCRIBE HISTORY` live in the same system, so an attempt can be correlated with the actual table version it targeted. Correlating a migration tool's log with a warehouse's history is usually a two-system problem; here it's a join.
- **Governance is inherited, not rebuilt.** Unity Catalog permissions, retention, and lineage apply to the audit trail exactly as they do to any other table. The team that controls access

to the data controls access to the record of what was done to it.

- **No second system to keep alive.** The audit trail cannot drift out of sync with the warehouse, or silently stop being written, because it is in the warehouse.

This is the argument for Databricks being load-bearing rather than incidental in a second sense. Delta supplies the *rollback*; the platform supplies the *observability surface the rollback gets recorded on* — for free, using tools a data team already runs.

---

## 5 · The privacy boundary

This began as the Buildathon's Noon Curveball and became the most substantial part of the system. The constraint: raw checkpoint intent must not reach an external service; the tool must still work when checkpoint fields are redacted or missing; and incomplete context must never be presented as complete.

> THE ASSUMPTION THAT BROKE
>
> Warden had been writing the full checkpoint explanation into `warden.migration_log.note` — a Databricks table, outside Entire's local boundary. The working assumption, *"more context in the log is better"*, was exactly wrong: a checkpoint can contain prompts and transcript text, and the log is the one place that content was crossing a trust boundary.

### Three mechanisms

#### CONTEXT_COMPLETENESS

`classify_completeness()` (`migrate.py:70`) returns `complete`, `redacted`, or `unavailable`. Redaction is detected via Entire's own marker shape — a regex matching bare `REDACTED` or `[REDACTED_<LABEL>]`, anchored so ordinary prose containing the word does not false-positive. The label is written to every log row and printed on every console run, so no downstream reader can mistake partial context for whole.

#### LOCALONLYTEXT

The important property here is that the guarantee is *structural* rather than conventional. Intent is wrapped in a small class whose `__str__` and `__repr__` return a placeholder, never the real text:

```
# the naive mistake — interpolating the wrapper into an outbound string
note = f"migration note: {intent}"
# → "migration note: <local-only text, use .reveal() explicitly>"
```

The real text is reachable only through an explicit `.reveal()`, which appears at exactly two genuinely local sites: the console print in `apply_migration`, and `console_text` in `describe_failure`. A future contributor who forgets the boundary gets a visibly wrong placeholder in a log row rather than a silent data leak — the failure mode changes from invisible to obvious.

ERROR_SUMMARY

This one was not in the brief. Reviewing the fix raised the question of where *else* the same shape of leak could occur, and the answer was already in the code: database error text. Some Delta constraint violations embed the offending row's column values directly in the message (`DELTA_VIOLATE_CONSTRAINT_WITH_VALUES`, raised on writes against an existing constraint). Forwarding a raw exception to the log would leak table data through a different door than the one just closed.

`error_summary()` reduces any exception to its type name before it crosses the boundary. The full message still prints locally for triage.

> VERIFIED LIVE, IN PRODUCTION DATA
>
> The demo's own failure raises `DELTA_NEW_CHECK_CONSTRAINT_VIOLATION` — the `ADD CONSTRAINT` variant, which reports only a row *count*, not values. So this specific run was never exposed to the value-embedding variant. `error_summary()` is a blanket guard for migrations that would hit it, not a reaction to this one. The logged row confirms the outcome: the note contains `ServerOperationError` and nothing else.

### What actually reaches Databricks

| FIELD | CONTENTS |
|---|---|
| `status` | `applied` / `healed` / `failed` |
| `checkpoint_id` | The identifier only — a pointer, never the content |
| `context_completeness` | `complete` / `redacted` / `unavailable` |
| `note` | Migration path, table, Delta version, exception *type*, and a pointer to the operator console |

Raw checkpoint intent and raw error messages appear in neither, in any code path — including the unrecoverable one.

---

## 6 · Entire integration

### Checkpoints

Warden calls `entire checkpoint list --json` and `entire checkpoint explain <id> --short` through `run_entire()`, a wrapper that never raises — every failure degrades to a warning and an explicit `unavailable` classification. A migration tool that crashed because a checkpoint could not be read would be worse than one that proceeds and says so.

`warden/resume_report.py` productises the read side. Given a checkpoint ID it reports completeness, the migration target, the latest safe log status, and a recommended next action — *without ever revealing checkpoint text*. It never calls `.reveal()`, and a test plants a secret to prove its output cannot carry one.

#### THE FRESH-SESSION CRITERION

Track 1 asks whether a fresh session can resume from these checkpoints. That was not staged here — it is how the Curveball response was produced. Commit `0d29a022c` deliberately ended the prior session at a stable point. A new session with no conversational memory began with `entire checkpoint explain 0d29a022c`, reconstructed the architecture, located the offending assumption, ran impact analysis before editing, and implemented the entire privacy boundary. Every commit from `cc16573cc` onward is that session's output.

### Graph

Impact analysis runs before the migration lands. The first real result was honest and unhelpful — `IMPACT DEGENERATE: warden.demo_users has no callers` — which is correct for a newly introduced symbol, and was reported rather than hidden.

The valuable result came from pointing the Graph at the functions the Curveball actually changed:

```
$ entire graph diff --base b8c9fe619 --head HEAD

warden/migrate.py (Python)
```

```
+ class LocalOnlyText added
+ function classify_completeness added
+ function error_summary added
~ function get_latest_checkpoint body changed (6 dependents)
~ function log_attempt signature changed  (2 dependents)
~ function apply_migration body changed (4 dependents)
```

That signature change *is* the privacy boundary: `log_attempt` gained the `context_completeness` parameter, and it is the single function that writes to the external service. Its two dependents are precisely the applied and healed call sites, each covered by a test — so the Graph's claimed blast radius and the suite's coverage agree. A direct impact query on `log_attempt` confirms the same two callers and one callee (`ensure_log_table`).

---

## 7 · Databricks integration

### Load-bearing

- **Delta time-travel is the rollback.** `RESTORE TABLE … TO VERSION AS OF` is the healing primitive. Without Delta's transaction log there is no version to restore to, and the product's central claim collapses.
- **Delta `CHECK` enforcement is the failure trigger.** Adding the constraint validates existing rows server-side at `ADD CONSTRAINT` time. The migration fails inside Delta, not inside our validation code — which is a stronger guarantee than a query we wrote.
- **A real Lakeview dashboard** over `migration_log`, created through the Lakeview REST API by `databricks/create_dashboard.py` — a workspace object, not a checked-in query file.

### Portable, and worth saying

The connector calls, the log table, and the seeding path would work against any SQL warehouse. Roughly the rollback and constraint mechanics are genuinely Delta-specific; the surrounding plumbing is not. Overstating that would be easy and wrong.

### Data provenance

Synthetic only. `databricks/seed.py` generates exactly 50 deterministic users on the reserved `example.invalid` domain (RFC 6761 — guaranteed never resolvable, so no real address can be implied). User 007 carries `plan='legacy'`, which is the single row that

violates the constraint, making the demo failure reproducible rather than incidental. No production or personal data is used anywhere.

## 8 · File reference

| FILE | LINES | ROLE |
|---|---|---|
| `warden/migrate.py` | 348 | The whole loop: checkpoint read, graph impact, apply, validate, heal, log. Contains `LocalOnlyText`, `classify_completeness`, `error_summary`, `describe_failure`. |
| `warden/resume_report.py` | 124 | Read-only readiness report from a checkpoint ID. Never reveals intent. |
| `warden/demo.sh` | 96 | One-command demo: pre-version → migrate → log row → post-version. `--self-test` runs with no credentials. |
| `warden/test_migrate.py` | 305 | Unit suite: checkpoint degradation, redaction classification, both leak boundaries, the unrecoverable heal path. |
| `warden/test_resume_report.py` | 36 | Proves a planted secret cannot appear in report output. |
| `warden/demo-recording.txt` | 58 | Captured successful live run — the fallback if the warehouse is unavailable. |
| `databricks/setup.sql` | 11 | The single Delta demo table. |
| `databricks/seed.py` | 87 | 50 synthetic rows with one deliberate offender. `--self-test` validates generation without a connection. |
| `databricks/create_dashboard.py` | 170 | Creates/updates the Lakeview dashboard via REST. Re-runnable; prints the workspace URL. |
| `databricks/test_integration.py` | 71 | Live-warehouse assertions; skips cleanly without credentials. |
| `databricks/dashboard_queries.sql` | 17 | Attempts vs. heals vs. failures per day. |
| `migrations/001_add_plan_column.sql` | 5 | The demo migration — adds a `CHECK` constraint on `plan`. |
| `migrations/001_validate.sql` | 10 | Returns offending rows; zero rows means pass. |

## 9 · Testing

**19 tests pass** without credentials (18 unit, plus 3 live-integration tests that skip cleanly); **21 with** credentials configured. The suite is deliberately weighted toward the boundaries rather than the happy path.

### What is actually pinned

- **Both leak boundaries.** Separate tests plant a secret intent and a secret row value, then assert neither reaches the outbound note — while confirming the console text *does* carry them, so the test cannot pass vacuously.
- **The unrecoverable path.** Migration fails *and* rollback fails: asserts the outcome is recorded `failed` not `healed`, that only exception type names cross the boundary, and that the error re-raises rather than being swallowed.
- **Checkpoint degradation.** Entire missing, malformed JSON, empty explain output, and redacted intent each classify correctly instead of crashing.
- **Schema evolution.** The log table's `ALTER` backfill ignores only a duplicate-column diagnostic and propagates genuine permission or connectivity failures.

> PROVEN NON-VACUOUS BY MUTATION
>
> The privacy test is only worth having if it fails when the boundary breaks. Replacing `error_summary(e)` with the raw exception in the unrecoverable branch makes it fail on the planted value; restoring the guard makes it pass. A passing assertion that cannot fail is not coverage.

### Review history

Two independent automated review passes were run against the codebase. The first found two real defects in this code, both fixed: an `ALTER TABLE` backfill that swallowed every exception (which could misclassify a healthy migration as failed and trigger an unnecessary rollback), and an `UnboundLocalError` when the version lookup itself failed. The second found that `demo.sh --self-test` could never fail — it ran without `errexit`, so a broken repository still reported success. Also fixed, and verified in both directions.

---

## 10 · Decisions & rejected options

### Pivoted from a checkpoint-mined onboarding tool

The original concept summarised checkpoint history into an onboarding briefing. It was rejected as safe but capped: it read checkpoints without ever *depending* on them for a decision. Warden makes the recovery itself checkpoint-driven, which is a stronger claim and a riskier build.

### A wrapper type instead of a coding convention

The first version of the privacy fix was a rule plus a regression test. That is enforcement by review, and review is exactly what fails on a busy day. `LocalOnlyText` makes the safe behaviour the default and the unsafe one deliberate.

### A multi-table scheduler was built and deliberately excluded

A bounded parallel scheduler and a health-report module were prototyped. Both passed unit tests. Both were left out of the submission because their accompanying seed change did not fully re-verify against the live warehouse before the deadline, and shipping an unverified change to the demo's own data path was the worse trade. A subsequent review pass then found several genuine defects in that layer — including an orchestration path that could roll back a migration which had actually succeeded — which retroactively confirmed the call.

### Type names, not sanitised messages

Rather than attempt to scrub row values out of error text, `error_summary()` discards the message entirely and keeps only the exception type. Partial sanitisation invites a pattern that misses a case; discarding the message cannot.

---

## 11 · Limits

- **One table, one migration, one validation path.** Deliberate, and the reason it works end-to-end.
- `RESTORE` **depends on retention.** If `VACUUM` has already purged the target version's files, the rollback fails outright. Production needs a retention policy that guarantees the rollback window survives.
- **Validation is a full-table scan.** At scale this must become partition-scoped or incremental.
- **Authentication is a long-lived personal token.** Production wants a service principal with short-lived OAuth credentials.

- `checkpoint_id` **is unsigned.** Anyone with warehouse write access could forge one. An HMAC over `(checkpoint_id, migration_name)`, keyed locally, would make a log row's checkpoint claim verifiable.
- **No access gate of its own.** Anyone holding the three env vars can apply a migration. A real deployment should require a reviewed checkpoint, turning checkpoint-*driven* into checkpoint-*authorised*.

---

## 12 · Roadmap

The ambition is a zero-downtime migration control plane for large, messy, bespoke schemas — the migrations a high-traffic platform cannot take offline for.

- **Multi-table dependency-aware parallelism.** Independent tables migrating concurrently under a declared worker bound; same-table work strictly ordered; a failure isolated to its own table rather than halting the run.
- **Live progress instead of a terminal pass/fail.** Rows processed, replay lag, current phase, warehouse spend — streamed, so an operator watches rather than waits. This is also what makes health alerts meaningful: a stall, lag past an SLO, or a rollback window approaching its retention limit are all mid-flight conditions.
- **An operator in the loop.** Inspect what validation caught, supply or correct offending rows, approve or hold a cutover. The checkpoint boundary already built is what makes exposing that safe — an agent can be shown *whether* context suffices without receiving the text.
- **Versioned cache reads plus CDF write catch-up.** Reads served from a stable version while the migration runs underneath; writes reconciled through Delta Change Data Feed or an outbox; compatibility-view cutover at the end. A cache alone only protects reads.
- **The migrations that actually frighten people.** Changing a primary key, splitting one table into several, merging several into one. These rewrite relationships rather than columns, so a blind revert is least acceptable exactly where it is most likely to be needed — and where "what was this trying to achieve?" is most expensive to reconstruct by hand.

---

## 13 · Verification evidence

Captured from a live Databricks Free Edition warehouse.

```
[warden] 'warden.demo_users' currently at Delta version 153
[warden] FAILURE: [DELTA_NEW_CHECK_CONSTRAINT_VIOLATION] 1 rows in
          workspace.warden.demo_users violate the new CHECK constraint
          (plan IN ('free', 'pro', 'enterprise'))
[warden] healing: restoring 'warden.demo_users' to Delta version 153 via time-travel…

── most recent row in warden.migration_log ──
status='healed'
context_completeness='complete'
note='migration failed (ServerOperationError); rolled back warden.demo_users
      to Delta version 153. Checkpoint intent and full error detail kept
      local-only (context: complete); see operator console for detail.'

── post-migration Delta version ──
Delta version: 153  ← matches pre-migration; the heal is real, not a printed message
```

Note what is absent from that row: no checkpoint intent, and no database error text beyond the exception's type name. That is the privacy boundary observable in production data rather than asserted in a document.

| CHECK | RESULT |
|---|---|
| Unit + integration suite, no credentials | `18 passed, 3 skipped` |
| Full suite with live credentials | `21 passed` |
| Credential-free self-test | `exit 0`; broken fixture → `exit 1` |
| Live demo end-to-end | `exit 0`, pre/post version match at 153 |
| Lakeview dashboard | `HTTP 200`, lifecycle `ACTIVE` |
| Fresh-environment checkpoint reconstruction | Recovered scope, files and next action from an ID alone |
| Repository secret scan | Clean; `.env` untracked and gitignored |

---

Warden · Entire Track 1 (Checkpoint-Native Developer Experience) · Best Use of Databricks opted in.

Built at the Bengaluru Tech Week Buildathon. Synthetic data throughout; no production or personal data.